Python For Data Engineering

```python
orders = [
    {"order_id": 1, "customer": "Ahmed", "amount": 500},
    {"order_id": 2, "customer": "Mohamed", "amount": 1200},
    {"order_id": 3, "customer": "Youssef", "amount": 300},
    {"order_id": 4, "customer": "Omar", "amount": 2000},
    {"order_id": 5, "customer": "Ali", "amount": 800}
]


# TASK 1 – Print every order
for order in orders:
    print(order)
print('='*33)


# TASK 2 – Print only the customer names
for order in orders:
    print(order['customer'])
print('='*33)


# TASK 3 – Print orders where amount > 1000
for order in orders:
    if order['amount'] > 1000:
        print(order['amount'])
print('='*33)


# TASK 4 – Calculate the total order amount
total_amount = 0

for order in orders:
    total_amount += order['amount']

print(f'total amounts: {total_amount}')     # total amounts: 4800
print('='*33)



# TASK 5 – Calculate the average order amount
total_amount = 0

for order in orders:
    total_amount += order['amount']

average_order_amount = total_amount / len(orders)
print(average_order_amount)     # 960.0
print('='*33)



# TASK 6 – Create a list containing only orders greater than 500
orders_greater_than_500 = []

for order in orders:
    if order['amount'] > 500:
        orders_greater_than_500.append(order)

print(orders_greater_than_500)
print('='*33)
```

```python
# TASK 7 — Add a "status" field to every order
# amount >= 1000 → "High"
# amount < 1000  → "Normal"
for order in orders:
    if order['amount'] >= 1000 :
        order['status'] = "High"
    else:
        order['status'] = "Normal"

for order in orders:
    print(order)
print('='*33)


# TASK 8 — Get only the names of High-value customers
high_value_customers = []

for order in orders:
    if order['status'] == "High":
        high_value_customers.append(order['customer'])

print(high_value_customers)
print('='*33)


# TASK 9 — Calculate total revenue from High-value orders
high_value_revenue = 0

for order in orders:
    if order['status'] == "High":
        high_value_revenue += order['amount']

print(high_value_revenue)       # 3200
print('='*33)


# TASK 10 — Find the order with the highest amount
highest_order  = orders[0]

for order in orders:
    if order['amount'] > highest_order ['amount']:
        highest_order = order

print(highest_order)    # {'order_id': 4, 'customer': 'Omar', 'amount': 2000, 'status': 'High'}
print('='*33)


# TASK 11 — Find the order with the lowest amount
lowest_order = orders[0]

for order in orders:
    if order['amount'] < lowest_order['amount']:
        lowest_order = order

print(lowest_order)     # {'order_id': 3, 'customer': 'Youssef', 'amount': 300, 'status': 'Normal'}
print('='*33)


# TASK 12 Create a list of customer names using list comprehension
customer_names = [
    order['customer']
    for order in orders
]

print(customer_names)       # ['Ahmed', 'Mohamed', 'Youssef', 'Omar', 'Ali']
print('='*33)
```

```python
# TASK 13 - Create a list containing only the order amounts
order_amounts = [
    order['amount']
    for order in orders
]

print(order_amounts)    # [500, 1200, 300, 2000, 800]
print('='*33)



# TASK 14 - Calculate total using sum()
total_sum = sum(
    order['amount']
    for order in orders
)

print(total_sum)        # 4800
print('='*33)


# TASK 15 - Calculate average using sum() and len()
average_amount = (
    sum(order['amount'] for order in orders) / len(orders)
)

print(average_amount)       # 960.0
print('='*33)


# TASK 16 - Count High and Normal orders
high_count = 0
normal_count = 0

for order in orders:
    if order['status'] == "High":
        high_count += 1
    else:
        normal_count += 1

print(f"high orders: {high_count}")     # high orders: 2
print(f"normal orders: {normal_count}")     # normal orders: 3
print('='*33)

# TASK 17 - Create a dictionary containing summary information
summary = {
    "total_orders": len(orders),
    "total_revenue": sum(order['amount'] for order in orders),
    "average_order": sum(order['amount'] for order in orders) / len(orders),
    "high_value_order": high_count,
    "normal_orders": normal_count
}
print(summary)
print('='*33)


# TASK 18 - Data Engineering Transformation
# Create a new dataset containing:
# order_id
# customer
# amount
# status
# Do not modify the original dataset.
transformed_orders = []

for order in orders:
    transformed_order = {
        "order_id": order['order_id'],
        "customer": order["customer"],
```

```
        "amount": order['amount'],
        "status": order['status']
    }

    transformed_orders.append(transformed_order)

print(transformed_orders)
print('='*33)
```

```
orders = [
    {"order_id": 1, "customer": "Ahmed", "amount": 500},
    {"order_id": 2, "customer": "Mohamed", "amount": 1200},
    {"order_id": 3, "customer": "Youssef", "amount": 300},
    {"order_id": 4, "customer": "Omar", "amount": 2000},
    {"order_id": 5, "customer": "Ali", "amount": 800}
]


# TASK 1 - Create a function that prints all orders
def print_orders(orders):
    for order in orders:
        print(order)

print_orders(orders=orders)
print('='*33)


# TASK 2 - Create a function that returns customer names
def get_customer_names(orders):
    names = []

    for order in orders:
        names.append(order['customer'])

    return names

customer_names = get_customer_names(orders)
print(customer_names)
print('='*33)


# TASK 3 - Create a function that returns orders where amount > 1000
def get_high_value_orders(orders):

    high_value_orders = []

    for order in orders:
        if order['amount'] > 1000:
            high_value_orders.append(order)

    return high_value_orders

high_value_orders = get_high_value_orders(orders)
print(high_value_orders)
print('='*33)


# TASK 4 - Create a function that calculates total revenue
def calculate_total_revenue(orders):
    total = 0

    for order in orders:
        total += order['amount']

    return total
```

```python
total = calculate_total_revenue(orders)
print(total)    # 4800
print('='*33)


# TASK 5 - Create a function that calculates average order value
def calculate_average_order_value(orders):
    total = 0

    for order in orders:
        total += order['amount']

    return total / len(orders)

average_order = calculate_average_order_value(orders)
print(average_order)        # 960.0
print('='*33)


# TASK 6 - Create a function that adds a status column
# amount >= 1000 → High
# amount < 1000  → Normal
def add_order_status(orders):
    for order in orders:
        if order['amount'] >= 1000:
            order['status'] = "High"
        else:
            order['status'] = "Normal"

    return orders

orders = add_order_status(orders)
print(orders)
print('='*33)


# TASK 7 - Create a function that validates orders
# An order is valid if:
# - order_id is not None
# - customer is not None
# - amount is not None
# - amount >= 0
def validate_orders(orders):
    valid_orders = []

    for order in orders:
        if (
            order['order_id'] is not None
            and order['customer'] is not None
            and order['amount'] is not None
            and order['amount'] >= 0
        ):
            valid_orders.append(order)

    return valid_orders

valid_orders = validate_orders(orders)
print(valid_orders)
print('='*33)


# TASK 8 - Create a function that calculates order count
def count_orders(orders):
    for order in orders:
        return len(orders)

order_count = count_orders(orders)
print(order_count)      # 5
print('='*33)
```

```python
# TASK 9 - Create a function that finds the highest-value order
def get_highest_order(orders):
    highest_order = orders[0]

    for order in orders:
        if order['amount'] > highest_order['amount']:
            highest_order = order

    return highest_order

highest_order = get_highest_order(orders)
print(highest_order)
print('=' * 33)


# TASK 10 - Create a function that finds the lowest-value order
def get_lowest_order(orders):
    lowest_order = orders[0]

    for order in orders:
        if order['amount'] < lowest_order['amount']:
            lowest_order = order

    return lowest_order

lowest_order = get_lowest_order(orders)
print(lowest_order)
print('=' * 33)

# TASK 11 - Create a function that filters orders based on a minimum amount
# Example:
# get_orders_by_min_amount(orders, 800)
def get_orders_by_min_amount(orders, minimum_amount):
    filtered_orders = []

    for order in orders:
        if order["amount"] >= minimum_amount:
            filtered_orders.append(order)

    return filtered_orders

filtered_orders = get_orders_by_min_amount(orders, 800)
print(filtered_orders)
print('=' * 33)

# TASK 12 - Create a function that calculates total revenue for a specific status
# Example:
# calculate_revenue_by_status(orders, "High")
def calculate_revenue_by_status(orders, status):
    total = 0

    for order in orders:
        if order["status"] == status:
            total += order["amount"]

    return total

high_revenue = calculate_revenue_by_status(
    orders,
    "High"
)
print("High Revenue:", high_revenue)
print('=' * 33)
```

```python
# TASK 13 - Create a reusable transformation function
# The function should:
# 1. Add status
# 2. Return the transformed dataset
def transform_orders(orders):
    transformed_orders = []

    for order in orders:
        transformed_order = order.copy()
        if transformed_order["amount"] >= 1000:
            transformed_order["status"] = "High"
        else:
            transformed_order["status"] = "Normal"
        transformed_orders.append(transformed_order)

    return transformed_orders

transformed_orders = transform_orders(orders)
print(transformed_orders)
print('=' * 33)

# ================================================

# TASK 14 - Build a complete mini ETL pipeline
# Extract
# Transform
# Validate
# Load
def extract_data():

    return [
        {"order_id": 1, "customer": "Ahmed", "amount": 500},
        {"order_id": 2, "customer": "Mohamed", "amount": 1200},
        {"order_id": 3, "customer": "Youssef", "amount": 300},
        {"order_id": 4, "customer": "Omar", "amount": 2000},
        {"order_id": 5, "customer": "Ali", "amount": 800}
    ]


def transform_data(orders):
    transformed_orders = []

    for order in orders:
        new_order = order.copy()
        if new_order["amount"] >= 1000:
            new_order["status"] = "High"
        else:
            new_order["status"] = "Normal"
        transformed_orders.append(new_order)

    return transformed_orders


def validate_data(orders):
    valid_orders = []

    for order in orders:
        if (
            order["order_id"] is not None
            and order["customer"] is not None
            and order["amount"] is not None
            and order["amount"] >= 0
        ):
            valid_orders.append(order)
    return valid_orders
```

```python
def load_data(orders):
    print("Loading data...")
    for order in orders:
        print(order)


# Run ETL Pipeline
raw_data = extract_data()
transformed_data = transform_data(raw_data)
valid_data = validate_data(transformed_data)
load_data(valid_data)

# ================================================

# TASK 15 — Build a complete Customer ETL pipeline
# Extract
# Transform
# Validate
# Load


# ============================================
# EXTRACT
# ============================================

def extract_data():
    return [
        {"customer_id": 1, "name": "Ahmed", "age": 25, "total_spent": 1500},
        {"customer_id": 2, "name": "Mohamed", "age": 35, "total_spent": 5000},
        {"customer_id": 3, "name": "Youssef", "age": 19, "total_spent": 800},
        {"customer_id": 4, "name": "Omar", "age": 42, "total_spent": 12000},
        {"customer_id": 5, "name": "Ali", "age": 28, "total_spent": 3000}
    ]


# ============================================
# TRANSFORM
# ============================================

def transform_data(customers):
    transformed_customers = []

    for customer in customers:
        new_customer = customer.copy()
        # Create age_group
        if new_customer["age"] < 25:
            new_customer["age_group"] = "Young"
        elif new_customer["age"] < 40:
            new_customer["age_group"] = "Adult"
        else:
            new_customer["age_group"] = "Senior"

        # Create customer_type
        if new_customer["total_spent"] >= 5000:
            new_customer["customer_type"] = "VIP"
        else:
            new_customer["customer_type"] = "Regular"
        transformed_customers.append(new_customer)
    return transformed_customers




# ============================================
# VALIDATE
# ============================================
```

```python
def validate_data(customers):
    valid_customers = []

    for customer in customers:
        if (
            customer["customer_id"] is not None
            and customer["name"] is not None
            and customer["age"] is not None
            and customer["total_spent"] is not None
            and customer["age"] >= 0
            and customer["total_spent"] >= 0
        ):
            valid_customers.append(customer)

    return valid_customers

# ==========================================
# LOAD
# ==========================================

def load_data(customers):
    print("Loading customer data...")
    for customer in customers:
        print(customer)


# ==========================================
# RUN ETL PIPELINE
# ==========================================
raw_data = extract_data()
transformed_data = transform_data(raw_data)
valid_data = validate_data(transformed_data)
load_data(valid_data)


# ================================================

# TASK 17 — Build a complete Sales ETL pipeline
# Extract
# Transform
# Validate
# Load


# ==========================================
# EXTRACT
# ==========================================

def extract_data():
    return [
        {
            "sale_id": 1,
            "product": "Laptop",
            "quantity": 2,
            "unit_price": 25000,
            "payment": "paid"
        },
        {
            "sale_id": 2,
            "product": "Mouse",
            "quantity": 5,
            "unit_price": 500,
            "payment": "pending"
        },
        {
            "sale_id": 3,
            "product": "Keyboard",
            "quantity": 3,
```

```
            "unit_price": 1200,
            "payment": "paid"
        },
        {
            "sale_id": 4,
            "product": "Monitor",
            "quantity": 2,
            "unit_price": 7000,
            "payment": "paid"
        },
        {
            "sale_id": 5,
            "product": "Headphones",
            "quantity": 4,
            "unit_price": 2000,
            "payment": "pending"
        }
    ]


# ==========================================
# TRANSFORM
# ==========================================

def transform_data(sales):
    transformed_sales = []

    for sale in sales:
        new_sale = sale.copy()

        # Calculate total amount
        new_sale["total_amount"] = (
            new_sale["quantity"] * new_sale["unit_price"]
        )

        # Create sales category
        if new_sale["total_amount"] >= 5000:
            new_sale["sales_category"] = "High"
        else:
            new_sale["sales_category"] = "Normal"
        # Create payment status
        if new_sale["payment"] == "paid":
            new_sale["payment_status"] = "Completed"
        else:
            new_sale["payment_status"] = "Pending"
        transformed_sales.append(new_sale)

    return transformed_sales


# ==========================================
# VALIDATE
# ==========================================
def validate_data(sales):
    valid_sales = []
    for sale in sales:
        if (
            sale["sale_id"] is not None
            and sale["product"] is not None
            and sale["quantity"] is not None
            and sale["quantity"] > 0
            and sale["unit_price"] is not None
            and sale["unit_price"] >= 0
            and sale["payment"] in ["paid", "pending"]
        ):
            valid_sales.append(sale)

    return valid_sales
```

```python
# ==========================================
# LOAD
# ==========================================
def load_data(sales):
    print("Loading sales data...")
    for sale in sales:
        print(sale)


# ==========================================
# RUN ETL PIPELINE
# ==========================================
raw_data = extract_data()
transformed_data = transform_data(raw_data)
valid_data = validate_data(transformed_data)
load_data(valid_data)

# ================================================

# TASK 18 — Build a complete Employee ETL pipeline
# Extract
# Transform
# Validate
# Load


# ==========================================
# EXTRACT
# ==========================================

def extract_data():
    return [
        {
            "employee_id": 1,
            "first_name": "Ahmed",
            "last_name": "Ali",
            "department": "Data",
            "salary": 18000,
            "status": "active"
        },
        {
            "employee_id": 2,
            "first_name": "Mohamed",
            "last_name": "Hassan",
            "department": "IT",
            "salary": 25000,
            "status": "active"
        },
        {
            "employee_id": 3,
            "first_name": "Youssef",
            "last_name": "Nady",
            "department": "Data",
            "salary": 9000,
            "status": "active"
        },
        {
            "employee_id": 4,
            "first_name": "Omar",
            "last_name": "Samir",
            "department": "HR",
            "salary": 12000,
            "status": "inactive"
        },
        {
            "employee_id": 5,
            "first_name": "Ali",
            "last_name": "Mostafa",
            "department": "Finance",
```

```
            "salary": 22000,
            "status": "active"
        }
    ]


# ==========================================
# TRANSFORM
# ==========================================

def transform_data(employees):
    transformed_employees = []

    for employee in employees:
        new_employee = employee.copy()

        # Create full name
        new_employee["full_name"] = (
            new_employee["first_name"]
            + " "
            + new_employee["last_name"]
        )

        # Create salary level
        if new_employee["salary"] >= 20000:
            new_employee["salary_level"] = "Senior"
        elif new_employee["salary"] >= 10000:
            new_employee["salary_level"] = "Mid"
        else:
            new_employee["salary_level"] = "Junior"

        # Create employment status
        if new_employee["status"] == "active":
            new_employee["employment_status"] = "Active"
        else:
            new_employee["employment_status"] = "Inactive"
        transformed_employees.append(new_employee)
    return transformed_employees


# ==========================================
# VALIDATE
# ==========================================

def validate_data(employees):
    valid_employees = []

    for employee in employees:
        if (
            employee["employee_id"] is not None
            and employee["first_name"] is not None
            and employee["last_name"] is not None
            and employee["department"] is not None
            and employee["salary"] is not None
            and employee["salary"] >= 0
            and employee["status"] in ["active", "inactive"]
        ):
            valid_employees.append(employee)
    return valid_employees


# ==========================================
# LOAD
# ==========================================

def load_data(employees):
    print("Loading employee data...")
    for employee in employees:
        print(employee)
```

```python
# ==========================================
# RUN ETL PIPELINE
# ==========================================
raw_data = extract_data()
transformed_data = transform_data(raw_data)
valid_data = validate_data(transformed_data)
load_data(valid_data)


# ================================================

# TASK 19 — Build a complete Product Inventory ETL pipeline
# Extract
# Transform
# Validate
# Load


# ==========================================
# EXTRACT
# ==========================================

def extract_data():

    return [
        {
            "product_id": 1,
            "product_name": "Laptop",
            "category": "Electronics",
            "quantity": 25,
            "unit_price": 25000
        },
        {
            "product_id": 2,
            "product_name": "Mouse",
            "category": "Electronics",
            "quantity": 8,
            "unit_price": 500
        },
        {
            "product_id": 3,
            "product_name": "Desk",
            "category": "Furniture",
            "quantity": 0,
            "unit_price": 7000
        },
        {
            "product_id": 4,
            "product_name": "Chair",
            "category": "Furniture",
            "quantity": 15,
            "unit_price": 3500
        },
        {
            "product_id": 5,
            "product_name": "Notebook",
            "category": "Stationery",
            "quantity": 50,
            "unit_price": 100
        }
    ]


# ==========================================
# TRANSFORM
# ==========================================
```

```python
def transform_data(products):
    transformed_products = []

    for product in products:
        new_product = product.copy()
        # Calculate inventory value
        new_product["inventory_value"] = (
            new_product["quantity"]
            * new_product["unit_price"]
        )

        # Create stock status
        if new_product["quantity"] == 0:
            new_product["stock_status"] = "Out of Stock"
        elif new_product["quantity"] <= 10:
            new_product["stock_status"] = "Low Stock"
        else:
            new_product["stock_status"] = "In Stock"

        # Create product category
        if new_product["category"] == "Electronics":
            new_product["product_category"] = "Tech"
        elif new_product["category"] == "Furniture":
            new_product["product_category"] = "Home"
        else:
            new_product["product_category"] = "Other"
        transformed_products.append(new_product)
    return transformed_products


# ============================================
# VALIDATE
# ============================================

def validate_data(products):
    valid_products = []
    for product in products:

        if (
            product["product_id"] is not None
            and product["product_name"] is not None
            and product["category"] is not None
            and product["quantity"] is not None
            and product["quantity"] >= 0
            and product["unit_price"] is not None
            and product["unit_price"] >= 0
        ):
            valid_products.append(product)
    return valid_products


# ============================================
# LOAD
# ============================================

def load_data(products):
    print("Loading inventory data...")
    for product in products:
        print(product)


# ============================================
# RUN ETL PIPELINE
# ============================================
raw_data = extract_data()
transformed_data = transform_data(raw_data)
valid_data = validate_data(transformed_data)
load_data(valid_data)
```